Software Engineering Department
Braude College of Engineering

Final Project – Phase A (61998)

A multi model deep learning system for predicting song popularity and suggesting audio enhancements

CODE: 26-1-D-5

By:
Arad Harush 211879606
Shahar Nahum 323964817

Advisor:
Dr. Dan Lemberg | lembergdan@braude.ac.il
Mrs. Elena Kramer | elenak@braude.ac.il

Link to GitHub:
https://github.com/aradharush/Song-Popularity-Prediction-System

1 Abstract

The modern music industry is characterized by an unprecedented volume of daily releases, making the task of identifying potential "hit" songs a significant challenge for producers and record labels. While recent advancements in Music Information Retrieval (MIR) have leveraged Deep Learning to predict song popularity, most existing solutions suffer from two major limitations: they rely on unimodal data (audio or metadata only) and operate as "black boxes," offering no actionable insights for improvement. This project proposes a comprehensive, dual-stage framework designed to bridge the gap between predictive analytics and creative optimization.

2 Introduction

The contemporary music industry is undergoing a paradigm shift driven by the rapid digitization of distribution channels. Streaming platforms have democratized music creation, resulting in an unprecedented volume of daily releases, estimated at over 100,000 new tracks every single day. This saturation has transformed the marketplace into a highly competitive environment where the ability to distinguish a potential "hit" from the vast majority of "misses" has become the Holy Grail for record labels, producers, and independent artists alike. Historically, the process of identifying commercially viable tracks relied heavily on the subjective intuition of A&R (Artists and Repertoire) professionals or basic statistical analysis of metadata. However, these traditional methods are increasingly insufficient in capturing the complex, stochastic nature of musical preference in the modern era, creating a pressing need for objective, data-driven analytical tools.

While the field of Music Information Retrieval (MIR) has seen significant advancements with the advent of Deep Learning, the current landscape of popularity prediction remains fragmented. A fundamental challenge lies in the multi-modal nature of music perception; the appeal of a song is derived from an intricate interplay between its sonic texture, rhythmic energy, and the semantic resonance of its lyrics. Most existing computational models fail to address this complexity, often focusing solely on audio signal processing or textual analysis in isolation. Furthermore, the industry faces a critical "interpretability gap." Conventional predictive models operate primarily as discriminative "black boxes"—they may successfully

classify a song's potential for success, but they offer no transparency regarding the underlying factors driving that decision. For a content creator, merely knowing that a track is predicted to achieve a low score is of limited value without actionable insights on how to rectify it.

To address these challenges, this project proposes a holistic framework that mirrors the cognitive process of a human music producer, architected as a dual-phase pipeline consisting of Perception and Optimization. In the Perception phase, the system replicates the listening experience by employing a Multi-Modal Neural Network. This network fuses three distinct data sources: raw audio signals (capturing production quality), lyrical content (capturing semantic meaning), and temporal metadata (specifically the song's release year) to account for shifting market trends. By synthesizing these inputs, the model achieves a nuanced understanding of the track's potential appeal. In the Optimization phase, the system transitions from a passive observer to an active advisor. Functioning as an autonomous "Virtual Producer," the framework utilizes advanced decision-making algorithms to analyze the track's feature profile. Instead of merely outputting a score, it identifies specific acoustic adjustments—such as modifying tempo or energy—that could bridge the gap between the current version and a potential hit, providing the user with a transparent and actionable roadmap for improvement.

3 Literature Review

The domain of Music Information Retrieval (MIR) has evolved significantly over the past decade, transitioning from handcrafted feature extraction to end-to-end Deep Learning (DL) methodologies. This section reviews the theoretical foundations and state-of-the-art technologies relevant to multi-modal popularity prediction and algorithmic optimization via Reinforcement Learning.

3.1 Multi-Modal Hit Song Prediction

Predicting the commercial success of music is a complex task due to the stochastic nature of public preference. While early approaches relied heavily on metadata or singular modalities, recent research demonstrates that combining auditory and textual data yields superior predictive performance. A pivotal study in this domain is the work of Castelli [1], which investigated the

efficacy of combining audio embeddings with lyric embeddings. Castelli's research highlights that a "Late Fusion" architecture—where features are extracted independently from audio and text before being concatenated—significantly outperforms uni-modal baselines. This finding aligns with broader research by Oramas et al. [2], who established that multi-modal networks integrating audio, text, and images provide a more comprehensive representation of musical content, leading to higher accuracy in classification and recommendation tasks.

3.2 Audio Analysis: CNNs and Spectrograms

To process raw audio data using deep learning, time-domain signals are typically converted into time-frequency representations, such as Mel-spectrograms, which can be processed as images. Choi et al. [3] demonstrated the effectiveness of Convolutional Neural Networks (CNNs) for various music tagging tasks, showing that CNNs can capture local temporal and harmonic patterns more effectively than traditional signal processing methods. A prominent architecture in this field is the Residual Network (ResNet), introduced by He et al. [4]. ResNet addresses the "vanishing gradient" problem in deep networks through the use of skip connections (residual blocks). By applying Transfer Learning from large-scale image recognition datasets (like ImageNet) to spectrogram analysis, ResNet models have become a standard for extracting high-level acoustic features, such as timbre and rhythm, from audio recordings.

3.3 Semantic Text Analysis: Transformers and BERT

Lyrical content plays a crucial role in a song's emotional resonance and market appeal. Traditional Natural Language Processing (NLP) methods, such as Bag-of-Words, often fail to capture context and semantic nuance. The introduction of BERT (Bidirectional Encoder Representations from Transformers) by Devlin et al. [5] marked a paradigm shift in NLP. Unlike previous models, BERT utilizes a mechanism of self-attention to understand the bidirectional context of words in a sentence. For tasks requiring sentence-level comparisons, Reimers and Gurevych [6] proposed Sentence-BERT (SBERT). This modification uses siamese network structures to derive semantically meaningful sentence embeddings that can be efficiently compared using cosine similarity, making it a highly effective tool for encoding the holistic meaning and sentiment of song lyrics.

3.4 Audio Feature Extraction for Tabular Modeling

While Deep Learning models like ResNet excel at processing raw spectrograms for classification, the optimization phase often requires a structured, interpretable state representation. To bridge the gap between continuous audio signals and the discrete feature space required by downstream tasks, specific musical descriptors must be extracted. McFee et al. [7] introduced Librosa, a comprehensive Python library for audio and music signal analysis, which has become the industry standard for this task. Librosa employs Digital Signal Processing (DSP) algorithms to extract both low-level features (e.g., Spectral Centroid, Zero-Crossing Rate) and high-level rhythmic features (e.g., Tempo/BPM, Beat frames). These scalar values serve as explicit inputs for optimization algorithms, allowing for the manipulation of tangible attributes of the song rather than abstract neural weights.

3.5 Optimization via Reinforcement Learning and Tabular DL

While prediction is a well-established field, algorithmic optimization of music features is an emerging area of research. This task is often framed as a Markov Decision Process (MDP) that can be solved using Deep Reinforcement Learning (DRL). The Deep Q-Network (DQN) algorithm, introduced by Mnih et al. [8], enables autonomous agents to learn optimal policies directly from high-dimensional state spaces, allowing the automation of complex decision-making processes. Furthermore, to model the environment in such optimization tasks, a robust and computationally efficient function approximator is required. In this work, we employ a Multi-Layer Perceptron (MLP) as the surrogate model. MLPs are well-established universal approximators capable of modeling complex, non-linear relationships between structured input features and target outputs. Their low inference latency makes them particularly suitable for serving as simulated environments in Reinforcement Learning loops, where rapid feedback is essential for the agent's iterative training process.

4 Project Overview

The music industry is highly competitive and saturated, making it increasingly difficult for artists and producers to predict the commercial success of a new track. Currently, decision-

making regarding song release and production often relies on intuition rather than data-driven insights.

To address this challenge, we developed a comprehensive predictive system designed to analyze and optimize song popularity potential. The system utilizes a multi-modal approach, ingesting three key data inputs: **raw audio files** (for acoustic analysis), **song lyrics** (for textual sentiment and thematic analysis), and **release year** (for trend context).

Upon processing these inputs, the system generates a **Popularity Score** ranging from 0 to 100, reflecting the predicted probability of the song's success in the current market. Beyond mere prediction, the system provides actionable value by offering algorithmic recommendations for improvement. These insights suggest specific modifications to the audio or lyrical content to maximize the song's potential score, effectively serving as an AI-powered production assistant for artists and producers.

4.1 System Requirements

The system is defined by the following functional and non-functional requirements, designed to ensure accuracy, usability, and performance.

4.1.1 Functional Requirements

- **Input Ingestion:** The system shall allow users to upload audio files (e.g., MP3, WAV) and input song lyrics and release year.
- **Data Processing:** The system shall convert raw audio into spectrogram representations and tokenize lyrics for NLP processing.
- **Popularity Prediction:** The system shall calculate and display a predicted popularity score (0-100) based on the processed inputs.
- **Recommendation Engine:** Based on the analysis, the system shall provide textual recommendations for improving the song's commercial potential (e.g., lyrical sentiment adjustment or acoustic modification).

4.1.2 Non-Functional Requirements

- **Performance:** The inference time (from upload to result display) shall not exceed 30 seconds under normal load.
- **Accuracy:** The model shall strive to achieve a low Mean Absolute Error (MAE) when comparing predicted popularity scores against ground-truth data from streaming services.
- **Reliability:** To ensure the optimization agent receives reliable feedback, the Pearson Correlation Coefficient between the popularity scores predicted by the surrogate MLP (on tabular features) and the primary ResNet model (on raw audio) must exceed 0.85 on the validation dataset.
- **Validity:** 100% of the feature modifications suggested by the Virtual Producer agent must fall within pre-defined, realistic musical ranges (e.g., BPM between 60–200, Energy between 0.0–1.0), ensuring no mathematically impossible values are presented to the user.
- **Latency:** To enable efficient Reinforcement Learning training, the surrogate environment (MLP Regressor) must perform a single inference step (state-to-reward calculation) in less than 50 milliseconds (0.05 seconds)..

4.2 Requirements Elicitation

The requirements for this system were derived primarily through a comprehensive **Literature Review**. We analyzed academic papers and existing studies in the fields of Music Information Retrieval (MIR) and Affective Computing.

This research highlighted a gap in current tools: while many studies focus on genre classification or mood detection, few offer a holistic approach that combines both audio and textual modalities to predict commercial success *and* provide actionable feedback. This understanding led to the requirement for a multi-modal system (Audio + Text) rather than relying on a single data source.

4.3 Research Process & Algorithm Design

Our research process focused on identifying the optimal method to represent a song as a mathematical object that a machine can "understand." We adopted a **Multi-Modal Deep Learning** approach, combining computer vision techniques for audio and natural language processing for lyrics.

Data Acquisition & Representation: The data set was constructed using data derived from **Spotify**.

- **Audio Representation:** We treat audio analysis as an image processing problem. Raw audio signals are converted into **Mel-Spectrograms**, which are visual representations of the frequency spectrum of a signal as it varies with time. This conversion allows us to utilize powerful Convolutional Neural Networks (CNNs).
- **Text Representation:** Lyrics are processed as sequential data, requiring advanced semantic understanding beyond simple keyword counting.

Model Architecture: The core of our system is an ensemble of three specialized components:

1. **ResNet50 (for Audio):** We utilize the ResNet50 architecture, a deep residual network pre-trained on ImageNet. The generated Spectrograms are fed into this network to extract high-level acoustic feature vectors (embeddings).
2. **BERT (for Lyrics):** For textual analysis, we employ BERT (Bidirectional Encoder Representations from Transformers). This state-of-the-art NLP model generates dense vector representations of the lyrics, capturing context, sentiment, and thematic nuance.
3. **MLP (Final Prediction):** The feature vectors from ResNet50 (audio) and BERT (text), along with the release year metadata, are concatenated into a single fused vector. This vector is fed into a **Multi-Layer Perceptron (MLP)** regressor, which outputs the final predicted popularity score.

4.4 Expected Challenges

Developing a predictive model for creative content involves inherent complexities. We have identified several key challenges:

- **Subjectivity vs. Generalization:** Music appreciation is subjective. However, our hypothesis is that "popularity" follows detectable patterns in the mass market. To mitigate the risk of bias toward a specific genre, our dataset is intentionally large and diverse, encompassing a wide variety of musical styles to ensure the model learns generalizable features rather than overfitting to a specific niche.

- **Computational Complexity:** The proposed architecture involves heavy computational loads, specifically the fine-tuning of large models like BERT and ResNet50. Training these models requires significant GPU resources. We aim to mitigate this by utilizing Transfer Learning (using pre-trained weights) and freezing early layers where possible to reduce the required training compute power.

- **Data Dimensionality:** High-dimensional data (spectrograms and large text embeddings) can lead to the "curse of dimensionality." Careful feature extraction and dimensionality reduction techniques within the MLP layers will be crucial to prevent overfitting.

4.5 Tools & Technologies

The system architecture is built upon a modern, open-source technology stack chosen to balance computational efficiency with ease of development and scalability. The selection of tools follows the industry standard for AI-integrated web applications.

- **Programming Language: Python 3.9+**

  Python was selected as the core language for the entire backend and logic layer due to its dominance in the Machine Learning ecosystem, extensive community support, and rich variety of libraries for audio signal processing.

- **Backend Framework: FastAPI**

  We utilize FastAPI for the server-side logic. This framework was chosen over traditional options (like Flask) for two primary reasons:

1. **Asynchronous Performance:** FastAPI supports asynchronous request handling (‘async‘/‘await‘), which is critical for our system where model inference can be time-consuming. It allows the server to handle multiple user requests concurrently without blocking.
2. **Automatic Documentation:** It automatically generates interactive API documentation (Swagger UI), facilitating easy testing and integration with the client side.

- **Frontend Framework: React.js**

  The user interface is developed using React. This library enables the creation of a dynamic Single Page Application (SPA). React's component-based architecture ensures a smooth user experience, allowing the page to update asynchronously (e.g., displaying the popularity score or audio graphs) without requiring a full page reload.

- **Machine Learning & Deep Learning Libraries:** The core intelligence of the system relies on the following Python libraries:

  - **PyTorch:** The primary deep learning framework used for constructing, training, and running the ResNet50 (audio) model. PyTorch was favored for its dynamic computation graph, which aids in debugging complex architectures.
  - **HuggingFace Transformers:** Utilized for implementing the pre-trained BERT model for lyrical sentiment analysis.
  - **Librosa:** The standard library for music and audio analysis. It is used in the preprocessing pipeline to load audio files and generate Mel-Spectrograms.
  - **Scikit-learn:** Employed for the final regression layer (MLP), data normalization, and evaluation metrics.

- **Database: MongoDB**

  A NoSQL database was selected to store user history and analysis results. Since the output of our analysis includes semi-structured data (variable-length recommendation lists and JSON metadata), MongoDB's flexible schema offers a significant advantage over rigid relational databases (SQL).

- **Development Environment:**

  Model training was conducted on **Google Colab Pro** to leverage NVIDIA T4 GPUs for accelerating the heavy computation required for Deep Learning. The source code is version-controlled and managed via **GitHub**.

4.6 GUI Design

4.6.1 Data Ingestion Panel

The entry point of the application features a drag-and-drop zone for audio files and structured input fields for metadata.

Audio Upload: Allows users to upload the raw track (MP3/WAV). Upon upload, a visual waveform is generated to confirm successful ingestion.

Lyrics Input: A multi-line text area where users paste the song's lyrics.

Metadata: A numeric input field for the song's release year, which is crucial for the context-aware prediction model.

"Analyze" Trigger: A central action button that initiates the backend inference pipeline (Feature Extraction $\rightarrow$ Prediction).

4.6.2 Results Dashboard

Once the analysis is complete, the user is presented with the "Current State" of the track and a list of suggested actions for improvement.

Popularity Score: Displaying the predicted popularity score (0-100) derived from the Multi-Modal Network.

The Improvement Recipe: A step-by-step list of specific actions generated by the agent.

Before/After Comparison: A side-by-side visualization showing how the suggested modifications shift the song's feature vector closer to the "Hit Cluster" in the latent space.

4.7 Methodology and System Architecture

4.7.1 Architectural Overview

The proposed solution is engineered as a dual-pipeline system designed to function sequentially. The architecture bridges the gap between high-dimensional unstructured data (audio/text) used for prediction, and structured tabular data used for optimization. The workflow consists of two main modules:

1. The Multi-Modal Predictive Module (Perception): A deep learning pipeline responsible for estimating the popularity score based on raw inputs.

2. The Optimization Module (Virtual Producer): A Reinforcement Learning (RL) loop responsible for suggesting feature modifications to improve the score.

4.7.2 Phase I: The Predictive Module

This module employs a "Late Fusion" strategy, processing each modality independently before combining them for the final regression task.

Audio Processing Branch:

Input: Raw MP3/WAV audio file.

Preprocessing: The audio is converted into a Mel-Spectrogram, a visual representation of the frequency spectrum over time.

Model: We utilize ResNet-50, a deep Convolutional Neural Network (CNN) pre-trained on ImageNet. The top layers are fine-tuned to extract a high-dimensional feature vector representing the song's timbre, rhythm, and production quality.

Lyrics Processing Branch:

Input: Raw text of the song's lyrics.

Model: We utilize Sentence-BERT (SBERT), a modification of the pre-trained BERT network. Unlike standard BERT which outputs token-level embeddings, SBERT uses siamese networks to derive a fixed-size vector that captures the semantic meaning and sentiment of the entire text.

Fusion and Regression:

* The embeddings from the Audio branch and the Text branch are concatenated along with the Release Year (normalized metadata).

* The combined vector is fed into a Multi-Layer Perceptron (MLP) regressor to output the final predicted popularity score ($P_{pred} \in [0, 100]$).

4.7.3 Phase II: The Optimization Module

Once a prediction is made, the system activates the optimization agent. Since manipulating raw spectrogram pixels is non-intuitive for music production, this module operates on interpretable, low-level audio features.

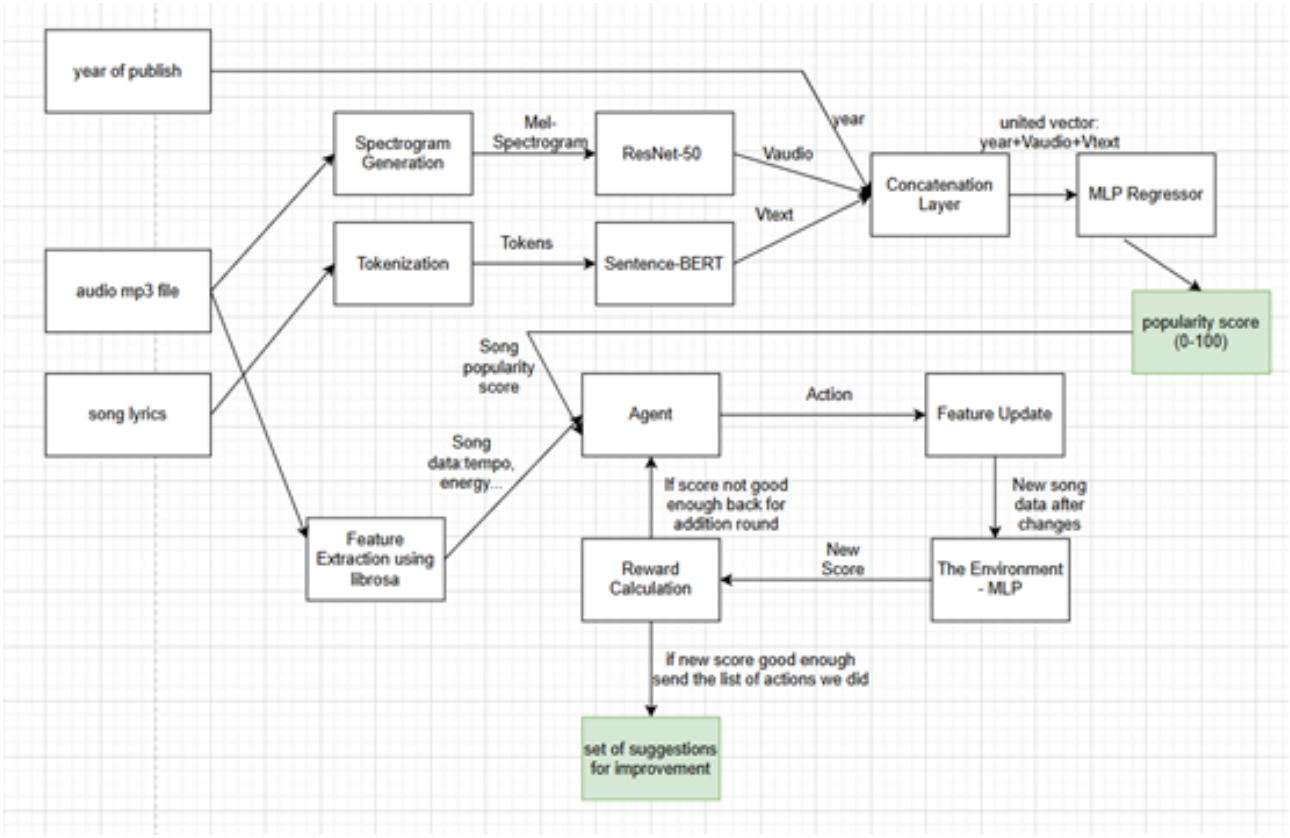
State Representation (Feature Extraction): Using the Librosa DSP library, the system extracts a vector of scalar features from the audio file (e.g., Tempo/BPM, Danceability, Energy, Loudness). This vector constitutes the initial "State" (S_0) for the RL agent.

The Environment (Surrogate Model): To simulate the reaction of the market to changes in the song without re-running the heavy ResNet model, we employ a Multi-Layer Perceptron (MLP) regressor. This MLP acts as a lightweight surrogate model, trained to approximate the main predictor's output with high fidelity while operating solely on the tabular features extracted by Librosa. Due to its low computational complexity, it serves as an efficient "Judge," providing immediate reward signals to the agent during the iterative optimization process.

The Agent (Deep Q-Network): We implement a DQN (Deep Q-Network) agent. The agent views the song as a state and possesses a discrete action space (e.g., Increase_Tempo, Decrease_Energy, No_Op). Its goal is to maximize the cumulative reward, defined as the positive difference in the popularity score predicted by MLP.

4.7.4 System Architecture Diagram:

The following diagram describes the entire system flow:



4.7.5 Algorithm Description: Optimization Process

The following pseudo-code describes the logic of the "Virtual Producer" agent during the inference phase (when a user uploads a song):

Algorithm 1 Song Optimization Agent (Inference)

Require: Audio (A), Target (T_{goal})
Ensure: Recommended Actions (R)

Step 1: Initialization
$S_{curr} \leftarrow \text{Extract}(A)$; $R \leftarrow []$; $Steps \leftarrow 0$
$P_{curr} \leftarrow \text{MLP.predict}(S_{curr})$
Step 2: Optimization Loop
while $P_{curr} < T_{goal}$ **and** $Steps <$ MAX_STEPS **do**
    $a \leftarrow \text{DQN.get\_action}(S_{curr})$ ▷ Choose best action
    $S_{next} \leftarrow \text{Apply}(S_{curr}, a)$ ▷ Apply mathematical modification
    $P_{next} \leftarrow \text{MLP.predict}(S_{next})$ ▷ Evaluate new state
    if $P_{next} > P_{curr}$ **then** ▷ Check for improvement
        Append a to R; $S_{curr} \leftarrow S_{next}$; $P_{curr} \leftarrow P_{next}$
    else
        break ▷ Stop if no improvement
    end if
    $Steps \leftarrow Steps + 1$
end while
return R, P_{curr}

4.8 Project Success Metrics

To rigorously assess whether the project has met its objectives, we have defined a set of Quantitative Key Performance Indicators (KPIs). These metrics are divided into three categories: the accuracy of the predictive model, the effectiveness of the optimization agent, and the overall system performance.

4.8.1 Predictive Model Accuracy (Perception Module)

The primary goal of the Multi-Modal Network (ResNet + BERT) is to predict song popularity more accurately than single-modality baselines.

Metric 1: Mean Absolute Error (MAE)

Definition: Measures the average magnitude of errors in the popularity prediction (scale 0-100), without considering their direction.

Success Criterion: The system must achieve an MAE < 12.0. This means, on average, the predicted score should not deviate from the actual Spotify popularity score by more than 12 points.

Formula: $MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$

Metric 2: Determination Coefficient (R^2 Score)

Definition: Represents the proportion of the variance for the dependent variable (popularity) that's explained by the independent variables (audio/text embeddings).

Success Criterion: $R^2 > 0.35$. While this may seem low in other domains, in hit-song prediction (a highly stochastic and subjective field), values above 0.3 are considered State-of-the-Art (SOTA) in academic literature.

Metric 3: Improvement over Baseline

Definition: Comparison of the Multi-Modal model against a simple MLP trained only on metadata or only on audio features.

Success Criterion: The Multi-Modal approach must show at least a 10% reduction in MSE (Mean Squared Error) compared to the uni-modal baselines, proving the hypothesis that fusing Audio+Lyrics adds value.

4.8.2 Optimization Agent Effectiveness (Action Module)

The goal of the RL Agent is to suggest meaningful improvements. Since there is no "ground truth" for optimization, we measure success based on the agent's ability to satisfy the surrogate model.

Metric 4: Surrogate Model Fidelity (Pearson Correlation)

Definition: We must verify that the "Judge" (MLP) used to train the agent actually agrees with the "Teacher" (ResNet/BERT). We measure the correlation between the scores output by the MLP (on tabular features) and the ResNet model (on raw audio) for the same test set.

Success Criterion: Pearson Correlation (r) > 0.85. This ensures the agent is learning from a reliable environment.

Metric 5: Average Score Uplift ($\Delta Score$)

Definition: The average increase in the predicted popularity score after applying the agent's suggested actions.

Success Criterion: For songs with an initial score < 50, the agent should be able to suggest actions that result in a predicted Uplift ≥ 15 points in the simulation environment.

Metric 6: Action Validity Rate

Definition: The percentage of suggested feature vectors that remain within the "realistic manifold" of music (e.g., ensuring Tempo doesn't exceed 200 BPM or Energy isn't negative).

Success Criterion: 100% of suggestions must be mathematically valid (within the min-max normalization bounds of the dataset).

4.8.3 System Performance (Non-Functional)

Metric 7: End-to-End Inference Latency

Definition: The total time elapsed from the moment a user clicks "Analyze" until the recommendations are displayed

Success Criterion: $<$ 30 seconds on a standard GPU-enabled server. This includes: Upload $\rightarrow$ Feature Extraction (Librosa) $\rightarrow$ Prediction $\rightarrow$ Agent Iterations $\rightarrow$ Output.

4.9 Testing and Validation Strategy

4.9.1 Overview of the Testing Plan

To ensure the reliability, accuracy, and robustness of the proposed system, a comprehensive testing strategy will be implemented. This strategy adopts a multi-layered approach, addressing three distinct aspects of the project:

1. Software Engineering Verification: Ensuring the functional correctness of data pipelines and feature extraction algorithms.

2. Machine Learning Evaluation: Rigorous statistical validation of the Predictive Module (ResNet+BERT) and the Surrogate Environment (MLP).

3. Agent Performance Assessment: Evaluating the effectiveness of the Reinforcement Learning policy in generating actionable and valid recommendations.

4.9.2 Phase I: Data Partitioning and Preprocessing Validation

Before training, the dataset (consisting of audio, lyrics, and metadata) will be split to prevent data leakage and ensure fair evaluation.

Data Splitting: The dataset will be partitioned into Training (80%), Validation (10%), and Testing (10%) sets using stratified sampling based on the popularity distribution. This ensures that all subsets maintain the same ratio of "hits" to "non-hits."

Sanity Checks (Unit Testing): Automated scripts will verify the integrity of the input data. Specifically, we will test the Librosa feature extraction pipeline to ensure that:

* No NaN (Not a Number) or infinite values are generated.

* Extracted features (e.g., BPM, Zero-Crossing Rate) fall within biologically and physically possible ranges.

4.9.3 Phase II: Predictive Model Evaluation (The Perception Module)

The core hypothesis of this research is that a multi-modal approach outperforms uni-modal baselines. To validate this, we will conduct an Ablation Study. We will train and test four distinct variations of the model using the Test Set:

1. Baseline A (Metadata Only): An MLP regressor trained solely on the song's release year and genre.

2. Baseline B (Audio Only): The ResNet-50 model trained only on Mel-spectrograms.

3. Baseline C (Lyrics Only): The Sentence-BERT model trained only on textual data.

4. Proposed Model (Late Fusion): The complete architecture fusing Audio, Text, and Metadata.

Success Criteria: The Proposed Model must demonstrate a statistically significant reduction in Mean Squared Error (MSE) and an increase in R^2 Score compared to all three baselines. This will empirically prove the value of the multi-modal fusion.

4.9.4 Phase III: Optimization Agent Evaluation (The Action Module)

Validating a generative agent is challenging due to the lack of "ground truth" (there is no single "correct" way to improve a song). Therefore, we will employ a two-step validation process:

Step 1: Surrogate Model Fidelity: Before trusting the agent, we must trust the environment it plays in. We will validate the MLP model (the "Judge") by comparing its predictions against the "Teacher" model (ResNet). We will calculate the Pearson Correlation Coefficient between the popularity scores predicted by the MLP (using tabular features) and ResNet (using raw audio) on the same test songs. A correlation above 0.85 is required to proceed.

Step 2: Policy Effectiveness (Simulation): We will run the trained agent on a batch of 500 low-popularity songs (Score < 40) from the test set. For each song, the agent will generate a sequence of actions. We will measure: * **Average Score Uplift:** The mean difference between the initial score and the final score after optimization

* **Action Validity:** A constraints check to ensure the agent does not suggest impossible values (e.g., negative energy or BPM > 250).

4.9.5 Phase IV: System Integration Testing (End-to-End)

The final phase validates the complete software pipeline as a user-facing product.

Latency Testing: We will measure the inference time for the full pipeline. The target is to process a 30-second audio clip (Upload $\rightarrow$ Feature Extraction $\rightarrow$ Prediction $\rightarrow$ Optimization Suggestion) in under 30 seconds on a GPU-enabled server.

Stress Testing: The system will be tested with varying file formats (MP3, WAV, FLAC) and varying audio qualities to ensure the feature extractor is robust to noise and compression artifacts.

User Acceptance Testing (UAT): A small focus group (or the project supervisors) will evaluate the interpretability of the text output. Specifically, we will assess whether the "Recipe" of changes (e.g., "Increase Tempo") is clearly understood and distinguishable from the technical metrics.

5 AI Tools

These are the AI tools we used during our work:

Google Scholar: Utilized to conduct a comprehensive literature review, locating foundational papers and state-of-the-art research in the field of Music Information Retrieval (MIR) and deep learning

ChatGPT: Served as a virtual coding assistant for debugging complex Python scripts, optimizing the logic of the Reinforcement Learning agent, and refining algorithm implementations.

Gemini: Assisted in synthesizing technical concepts, drafting high-level academic explanations, and structuring the project's documentation and theoretical background.

Draw.io: Employed for designing detailed system architecture diagrams and flowcharts, visually mapping the end-to-end pipeline from data ingestion to the optimization output.

6 References

1. Castelli, E. (2021). Hit Song Prediction system based on audio and lyrics embeddings. Master's Thesis, Politecnico di Milano.

2. Oramas, S., Nieto, O., Barbieri, F., & Serra, X. (2017). Multi-modal music genre classification with audio, text and images. In Proceedings of the 18th ISMIR Conference, 168-175.

3. Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2017). Convolutional recurrent neural networks for music classification. In 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2392-2396.

4. He, K., Zhang, X., Ren, S., & Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770-778.

5. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.

6. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.

7. McFee, B., Raffel, C., Liang, D., Ellis, D. P., McVicar, M., Battenberg, E., & Kelley, O. (2015). librosa: Audio and music signal analysis in python. In Proceedings of the 14th python in science conference, 18-25.

8. Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.